

✓ CS 280: Flow Matching

You'll be implementing your own flow matching model with an attention-based UNet. This project was developed by [Jorge Diaz Chao](#), [Jiaxin Ge](#), [Angjoo Kanazawa](#) and [Alexei A. Efros](#), and builds on top of the [work](#) of [Tabrizi R.](#) et al. (2024).

```
import math
import os

import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from tqdm import tqdm
import matplotlib.pyplot as plt
```

```
if torch.cuda.is_available():
    device = torch.device('cuda')
elif torch.backends.mps.is_available():
    device = torch.device('mps')
else:
    device = torch.device('cpu')
print(f"Using device: {device}")
```

```
Using device: cuda
```

```
C, H, W = 1, 32, 32
BATCH_SIZE = 64
```

✓ Load the Dataset

We first import the MNIST dataset. As a bonus, you could try other datasets like CIFAR-10 or Celeb-A. We resize images from 28×28 to 32×32, so they work nicely with the ideal flow machines you'll be implementing later in [idealfow.ipynb](#).

```
def load_mnist(train=True):
    transform = transforms.Compose([
        transforms.Resize((H, W)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.5], std=[0.5]),
```

```

    ])
    return torchvision.datasets.MNIST(
        root='./data', train=train, download=True, transform=transform
    )

def denormalize(x):
    return x * 0.5 + 0.5

train_dataset = load_mnist(train=True)
eval_dataset = load_mnist(train=False)
dataloader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=False)

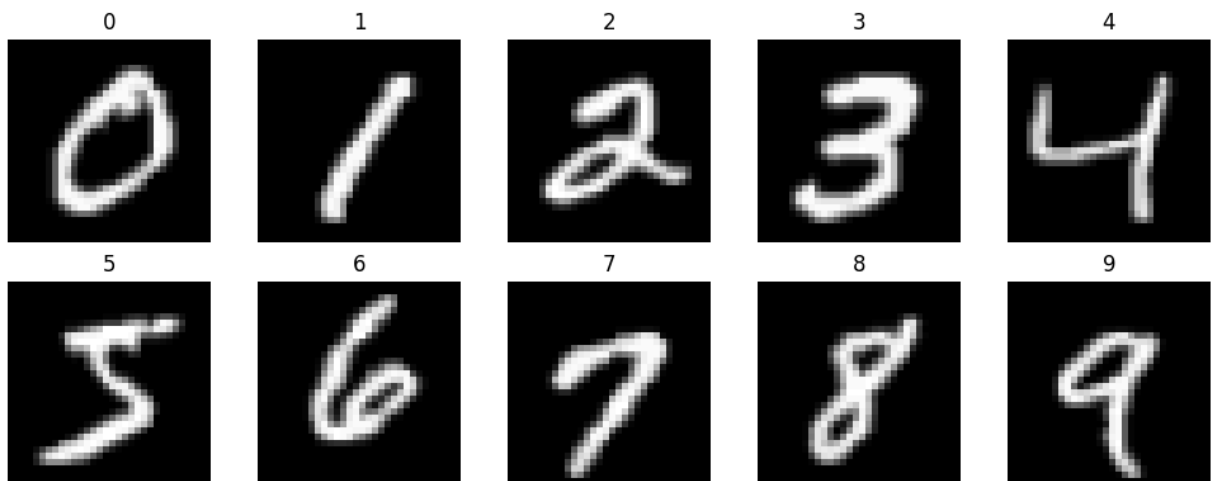
examples = {}
for x, y in train_dataset:
    if y not in examples:
        examples[y] = x
    if len(examples) == 10:
        break

fig, axes = plt.subplots(2, 5, figsize=(10, 4))

for digit, ax in zip(range(10), axes.flatten()):
    x = denormalize(examples[digit]).squeeze(0)
    ax.imshow(x, cmap="gray")
    ax.set_title(f"{digit}")
    ax.axis("off")

plt.tight_layout()
plt.show()

```



✓ Building the UNet

We'll start off by implementing a denoising UNet, which consists of residual blocks, attention blocks, downsampling and upsampling convolutions with skip connections.

We first implement the residual block (`ResBlock`), a skip-connected convolutional block. The `ResBlock` learns a residual (change) to the features instead of a full transformation, which improves gradient flow, enables deeper networks and preserves low-level information while refining it.

The diagram below shows the `ResBlock` architecture you should implement, the blocks should provide information for what to initialize, while their connections inform how you should construct the forward function.

Conv (a, b, c, d) – `nn.Conv2d(in_channels=a, out_channels=b, kernel_size=c, padding=d)`

G-Norm (a, b) – `nn.GroupNorm(num_groups=a, num_channels=b)`

Silu – `F.silu`



```
class ResBlock(nn.Module):
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
        self.norm1 = nn.GroupNorm(32, in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)
        self.norm2 = nn.GroupNorm(32, out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1)
        self.skip = (
            nn.Conv2d(in_channels, out_channels, kernel_size=1, padding=0)
            if in_channels != out_channels
            else nn.Identity()
        )

    def forward(self, x: torch.Tensor):
        h = F.silu(self.norm1(x))
        h = self.conv1(h)
        h = F.silu(self.norm2(h))
        h = self.conv2(h)
        return h + self.skip(x)
```

Then, we implement the attention block (`AttnBlock`), which utilizes self-attention so every token can incorporate global context from the entire feature map. It adds this refinement back through a residual connection for stable learning. Like the `ResBlock`, the diagram below shows how the `AttnBlock` should be constructed.

Attn (a, b) – `nn.MultiheadAttention(embed_dim=a, num_heads=b)`



```
class AttnBlock(nn.Module):
    def __init__(self, channels: int, num_heads: int = 4):
        super().__init__()
        self.norm = nn.GroupNorm(32, channels)
        self.attn = nn.MultiheadAttention(embed_dim=channels, num_heads=num_heads)

    def forward(self, x: torch.Tensor):
        B, C, H, W = x.shape
        h = self.norm(x)
        h = h.view(B, C, H * W).permute(0, 2, 1)
        h, _ = self.attn(h, h, h)
        h = h.permute(0, 2, 1).view(B, C, H, W)
        return x + h
```

Following the diagram below, we finalize our first **UNet**. Note that **DownConv** and **UpConv** are different from regular convolutions we utilized in the **ResBlock**, which downsample and upsample the hidden features and are responsible for the U shape of the network.

DownConv (a, b, c, d) – `nn.Conv2d(in_channels=a, out_channels=b, kernel_size=c, stride=2, padding=d)`

UpConv (a, b, c, d) – `nn.Upsample(scale_factor=2)` + `nn.Conv2d(in_channels=a, out_channels=b, kernel_size=c, padding=d)`

ResBlock (a, b) – `ResBlock(in_channels=a, out_channels=b)`

AttnBlock (a, b) – `AttnBlock(channels=a, num_heads=b)`

Cat – `torch.cat`



```
class UNet(nn.Module):
    def __init__(
        self,
        in_channels: int = 1,
        hidden_dim: int = 32
    ):
        super().__init__()
        D = hidden_dim

        self.conv1 = nn.Conv2d(in_channels, D, kernel_size=3, padding=1)
```

```

self.res1 = ResBlock(D, D)
self.attn1 = AttnBlock(D, num_heads=4)

self.downconv1 = nn.Conv2d(D, D, kernel_size=3, stride=2, padding=1)
self.res2 = ResBlock(D, 2 * D)
self.attn2 = AttnBlock(2 * D, num_heads=4)

self.downconv2 = nn.Conv2d(2 * D, 2 * D, kernel_size=3, stride=2, padding=1)
self.res3 = ResBlock(2 * D, 4 * D)

self.mid_res1 = ResBlock(4 * D, 4 * D)
self.mid_attn = AttnBlock(4 * D, num_heads=4)
self.mid_res2 = ResBlock(4 * D, 4 * D)

self.up2_res = ResBlock(4 * D, 4 * D)
self.up2_conv = nn.Sequential(
    nn.Upsample(scale_factor=2),
    nn.Conv2d(4 * D, 4 * D, kernel_size=3, padding=1),
)

self.up1_res = ResBlock(4 * D + 2 * D, 2 * D)
self.up1_attn = AttnBlock(2 * D, num_heads=4)
self.up1_conv = nn.Sequential(
    nn.Upsample(scale_factor=2),
    nn.Conv2d(2 * D, 2 * D, kernel_size=3, padding=1),
)

self.out_res = ResBlock(2 * D + D, D)
self.out_attn = AttnBlock(D, num_heads=4)
self.out_norm = nn.GroupNorm(32, D)
self.out_conv = nn.Conv2d(D, in_channels, kernel_size=3, padding=1)

def forward(self, x: torch.Tensor):
    h = self.conv1(x)
    h = self.res1(h)
    skip1 = self.attn1(h)

    h = self.downconv1(skip1)
    h = self.res2(h)
    skip2 = self.attn2(h)

    h = self.downconv2(skip2)
    h = self.res3(h)

    h = self.mid_res1(h)
    h = self.mid_attn(h)
    h = self.mid_res2(h)

    h = self.up2_res(h)
    h = self.up2_conv(h)

```

```

        h = torch.cat([h, skip2], dim=1)
        h = self.up1_res(h)
        h = self.up1_attn(h)
        h = self.up1_conv(h)

        h = torch.cat([h, skip1], dim=1)
        h = self.out_res(h)
        h = self.out_attn(h)

        h = F.silu(self.out_norm(h))
        return self.out_conv(h)

```

✓ Training a Denoising UNet

We aim to solve the following denoising problem, i.e. given a noisy image x_1 , we aim to train a denoiser D_θ such that it maps z to a clean image x . To do so, we can optimize over an L2 loss

$$L = \mathbb{E}_{z,x} \|D_\theta(z) - x\|^2.$$

To train our denoiser, we need to generate training data pairs of (z, x) , where each x is a clean MNIST digit. For each training batch, we can produce z from x such that

$$z = x + \sigma\epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, I).$$

Visualize the different noising processes over $\sigma = \{0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}$, assuming normalized $x \in [-1, 1]$.

```

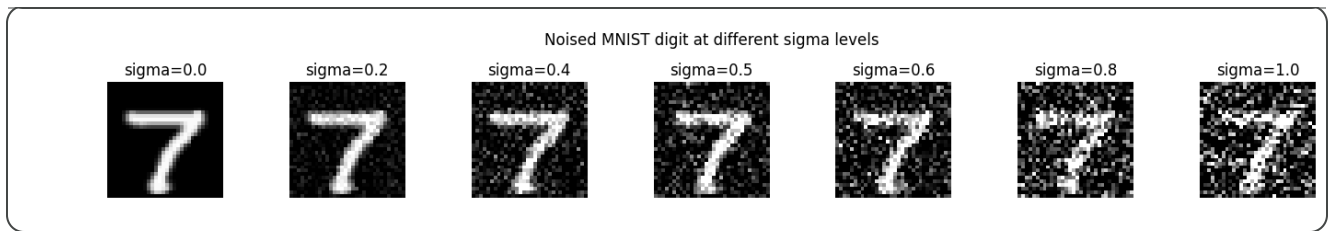
sigmas = [0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0]

x_sample, _ = next(iter(DataLoader(eval_dataset, batch_size=1, shuffle=True)))

fig, axes = plt.subplots(1, len(sigmas), figsize=(14, 2))
for ax, sigma in zip(axes, sigmas):
    eps = torch.randn_like(x_sample)
    z = x_sample + sigma * eps
    img = denormalize(z[0]).squeeze().clamp(0, 1)
    ax.imshow(img, cmap='gray')
    ax.set_title(f'sigma={sigma}')
    ax.axis('off')

plt.suptitle('Noised MNIST digit at different sigma levels')
plt.tight_layout()
os.makedirs('figures', exist_ok=True)
plt.savefig('figures/noise_visualization.png', dpi=100, bbox_inches='tight')
plt.show()

```



Now, we'll train our model to perform denoising. Particularly,

- Train a denoiser to denoise noisy image z with $\sigma = 0.5$ applied to a clean image x .
- Train on the MNIST training set for 5 or more epochs. Shuffle the dataset before initializing the dataloader. You should add noise to the images after being fetched from the dataloader to improve generalization.
- Your denoiser should be the UNet network you recently implemented with recommended `hidden_dim=32`.
- For the optimizer, utilize `torch.optim.Adam` optimizer with learning rate of 10^{-4} .

Deliverables

- Plot the training loss curve over iterations.
- Show sample denoised results on eval set digits after epochs 1 and 5.

```
SIGMA = 0.5
NUM_EPOCHS = 5

model_denoiser = UNet(in_channels=C, hidden_dim=32).to(device)
optimizer_denoise = torch.optim.Adam(model_denoiser.parameters(), lr=1e-4)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)

denoise_losses = []
denoise_snapshots = {}

for epoch in range(1, NUM_EPOCHS + 1):
    model_denoiser.train()
    for x, _ in tqdm(train_loader, desc=f'Denoiser epoch {epoch}/{NUM_EPOCHS}'):
        x = x.to(device)
        eps = torch.randn_like(x)
        z = x + SIGMA * eps
        x_hat = model_denoiser(z)
        loss = F.mse_loss(x_hat, x)
        optimizer_denoise.zero_grad()
        loss.backward()
        optimizer_denoise.step()
        denoise_losses.append(loss.item())

    if epoch in (1, 5):
        model_denoiser.eval()
        with torch.no_grad():
```

```

x_eval, _ = next(iter(DataLoader(eval_dataset, batch_size=10, shuffle=True)))
x_eval = x_eval.to(device)
z_eval = x_eval + SIGMA * torch.randn_like(x_eval)
x_hat_eval = model_denoiser(z_eval)
denoise_snapshots[epoch] = (z_eval.cpu(), x_hat_eval.cpu(), x_eval.cpu())

os.makedirs('checkpoints', exist_ok=True)
torch.save(model_denoiser.state_dict(), 'checkpoints/denoiser.pt')

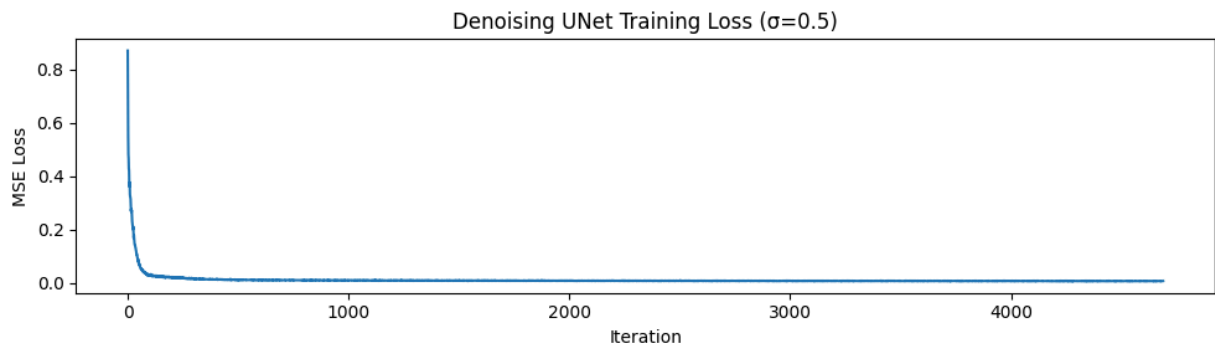
plt.figure(figsize=(10, 3))
plt.plot(denoise_losses)
plt.xlabel('Iteration')
plt.ylabel('MSE Loss')
plt.title('Denoising UNet Training Loss ( $\sigma=0.5$ )')
plt.tight_layout()
plt.savefig('figures/denoiser_loss.png', dpi=100)
plt.show()

```

```

Denoiser epoch 1/5: 100%|██████████| 938/938 [00:32<00:00, 28.67it/s]
Denoiser epoch 2/5: 100%|██████████| 938/938 [00:32<00:00, 28.75it/s]
Denoiser epoch 3/5: 100%|██████████| 938/938 [00:32<00:00, 28.75it/s]
Denoiser epoch 4/5: 100%|██████████| 938/938 [00:32<00:00, 28.74it/s]
Denoiser epoch 5/5: 100%|██████████| 938/938 [00:32<00:00, 28.73it/s]

```



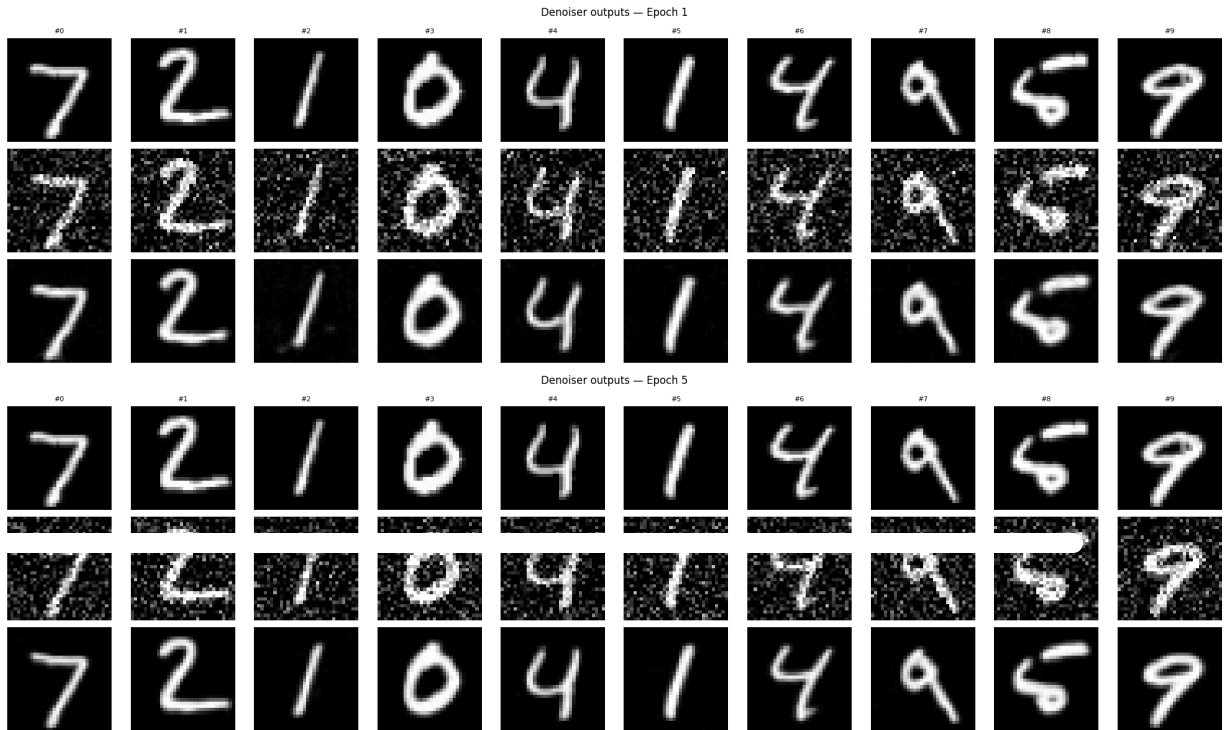
Visualize denoised results on test set digits at the end of epochs 1 and 5.

```

for epoch, (z, x_hat, x_clean) in sorted(denoise_snapshots.items()):
    n = z.shape[0]
    fig, axes = plt.subplots(3, n, figsize=(2 * n, 6))
    row_labels = ['Clean', 'Noisy (z)', 'Denoised ( $\hat{x}$ )]
    for i in range(n):
        for row, imgs in enumerate([x_clean, z, x_hat]):
            axes[row, i].imshow(denormalize(imgs[i]).squeeze().clamp(0, 1), cmap=gray)
            axes[row, i].axis('off')
        axes[0, i].set_title(f'#{i}', fontsize=8)
    for row, lbl in enumerate(row_labels):
        axes[row, 0].set_ylabel(lbl, fontsize=9)
    plt.suptitle(f'Denoiser outputs - Epoch {epoch}')
    plt.tight_layout()

```

```
plt.savefig(f'figures/denoiser_epoch{epoch}.png', dpi=100)
plt.show()
```



Our denoiser was trained on MNIST digits noised with $\sigma = 0.5$. How would it perform on out-of-distribution noise levels? Visualize the denoiser results on test set digits with $\sigma = \{0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}$.

```
sigmas_ood = [0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0]
n_show = 5

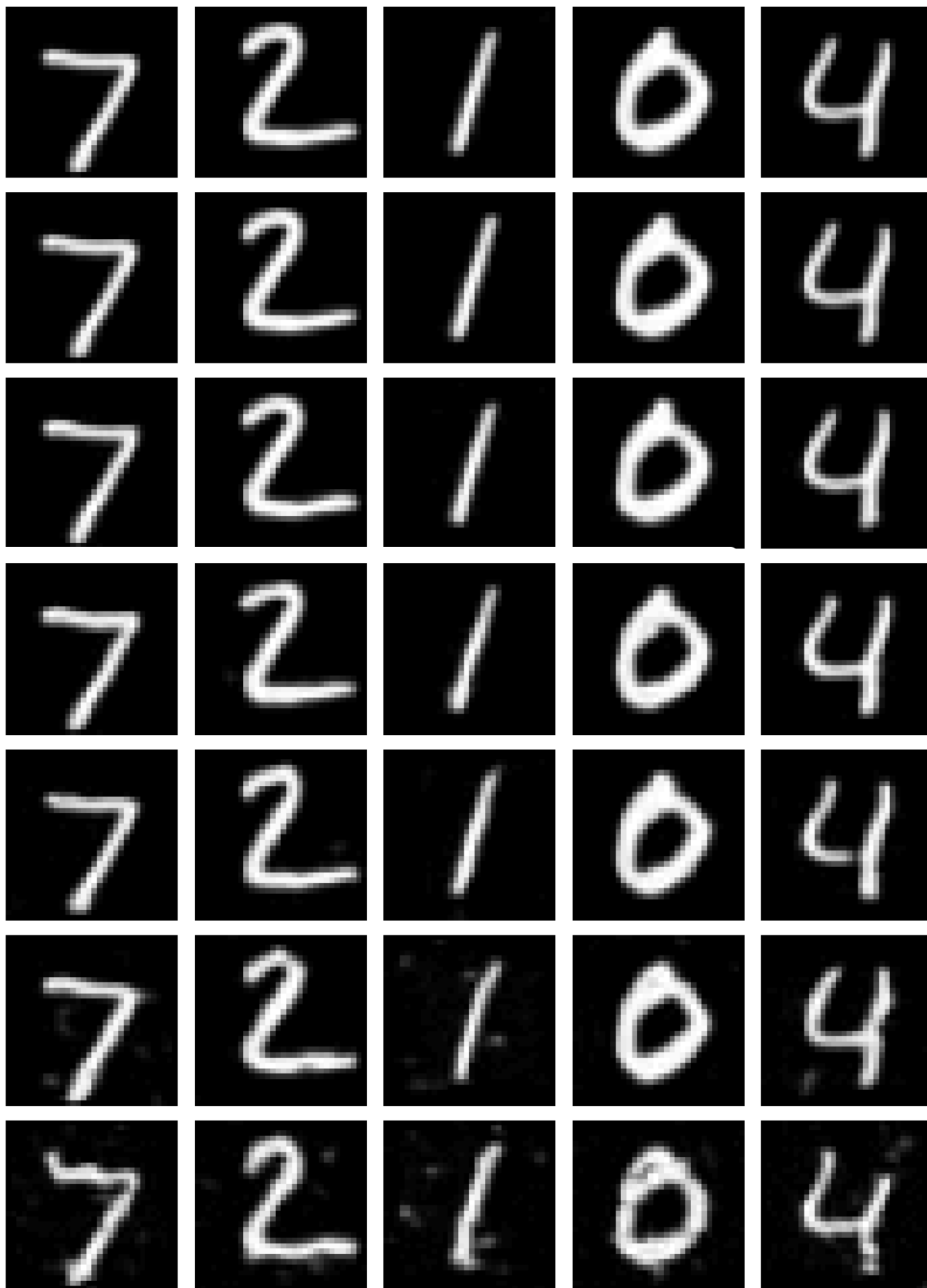
x_eval_ood, _ = next(iter(DataLoader(eval_dataset, batch_size=n_show, shuffle=False)))
x_eval_ood = x_eval_ood.to(device)

model_denoiser.eval()
fig, axes = plt.subplots(len(sigmas_ood), n_show, figsize=(2 * n_show, 2 * len(sigmas_ood)))

for row, sigma_ood in enumerate(sigmas_ood):
    with torch.no_grad():
        z_ood = x_eval_ood + sigma_ood * torch.randn_like(x_eval_ood)
        x_hat_ood = model_denoiser(z_ood)
    for col in range(n_show):
        axes[row, col].imshow(denormalize(x_hat_ood[col].cpu()).squeeze().clamp(0, 1))
        axes[row, col].axis('off')
    axes[row, 0].set_ylabel(f' $\sigma={sigma_ood}$ ', fontsize=9)

plt.suptitle('Denoiser (trained  $\sigma=0.5$ ) on OOD noise levels', y=1.01)
plt.tight_layout()
```

```
plt.savefig('figures/ood_eval.png', dpi=100, bbox_inches='tight')  
plt.show()
```

Denoiser (trained $\sigma=0.5$) on OOD noise levels

In order to make denoising generative, we'd like to be able to denoise pure Gaussian noise. We can think of this as starting with a blank canvas $z = \epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$ and denoising it to get a clean image x . We'll first attempt the naive approach, training our network like we just did but start with pure noise $\epsilon \sim \mathcal{N}(0, I)$ instead of a noised digit from the dataset. Again, display results for different epochs.

On top of that, compute the average image of the training set. What do you notice when comparing the average image and our attempts to denoise pure noise? Why would it show such behavior?

```
NUM_EPOCHS = 5

model_pure = UNet(in_channels=C, hidden_dim=32).to(device)
optimizer_pure = torch.optim.Adam(model_pure.parameters(), lr=1e-4)

pure_losses = []
pure_snapshots = {}

for epoch in range(1, NUM_EPOCHS + 1):
    model_pure.train()
    for x, _ in tqdm(train_loader, desc=f'Pure-noise epoch {epoch}/{NUM_EPOCHS}'):
        x = x.to(device)
        eps = torch.randn_like(x)
        x_hat = model_pure(eps)
        loss = F.mse_loss(x_hat, x)
        optimizer_pure.zero_grad()
        loss.backward()
        optimizer_pure.step()
        pure_losses.append(loss.item())

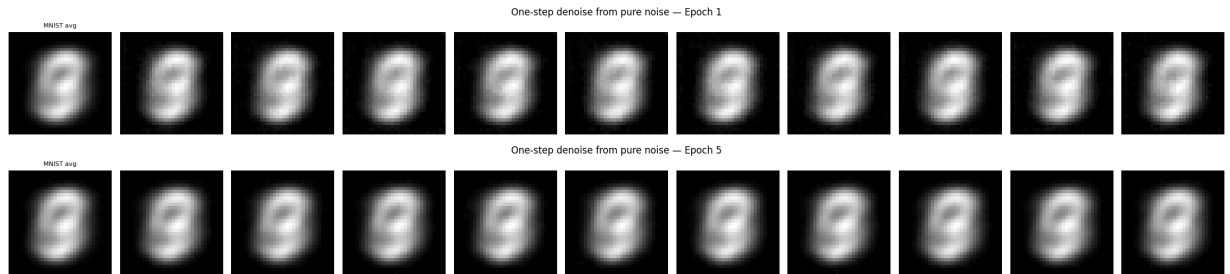
    if epoch in (1, 5):
        model_pure.eval()
        with torch.no_grad():
            eps_test = torch.randn(10, C, H, W, device=device)
            pure_out = model_pure(eps_test).cpu()
            pure_snapshots[epoch] = pure_out

all_imgs = torch.stack([train_dataset[i][0] for i in range(len(train_dataset))])
avg_image = all_imgs.mean(0)

for epoch, imgs in sorted(pure_snapshots.items()):
    fig, axes = plt.subplots(1, 11, figsize=(22, 2.5))
    axes[0].imshow(denormalize(avg_image).squeeze().clamp(0, 1), cmap='gray')
    axes[0].set_title('MNIST avg', fontsize=8)
    axes[0].axis('off')
    for i in range(10):
        axes[i + 1].imshow(denormalize(imgs[i]).squeeze().clamp(0, 1), cmap='gr
        axes[i + 1].axis('off')
```

```
plt.suptitle(f'One-step denoise from pure noise – Epoch {epoch}')
plt.tight_layout()
plt.savefig(f'figures/pure_noise_epoch{epoch}.png', dpi=100)
plt.show()
```

```
Pure-noise epoch 1/5: 100%|██████████| 938/938 [00:32<00:00, 28.65it/s]
Pure-noise epoch 2/5: 100%|██████████| 938/938 [00:32<00:00, 28.73it/s]
Pure-noise epoch 3/5: 100%|██████████| 938/938 [00:32<00:00, 28.72it/s]
Pure-noise epoch 4/5: 100%|██████████| 938/938 [00:32<00:00, 28.71it/s]
Pure-noise epoch 5/5: 100%|██████████| 938/938 [00:32<00:00, 28.72it/s]
```



Written response – Average image vs. one-step denoising from pure noise

When we train a one-step denoiser to map pure Gaussian noise directly to a clean digit, the outputs closely resemble the *average MNIST image*: a blurry, gray silhouette that superimposes all ten digit classes. This happens for a fundamental reason rooted in the L2 loss.

The optimal predictor under mean squared error is the *conditional mean* $\mathbb{E}[x \mid z]$. Because pure noise $\varepsilon \sim \mathcal{N}(0, I)$ carries **no class information whatsoever**, the conditional mean collapses to the *unconditional mean* of the entire training distribution: $\mathbb{E}[x]$. The model cannot tell which digit to generate—every noise sample looks the same to it—so it hedges by outputting the average of all possible digits, which is exactly what we see.

The MNIST dataset average is a blurred superposition of all handwritten digits, and our one-step denoiser from pure noise reproduces precisely that blurring. This shows why iterative denoising (e.g. flow matching) is necessary: by conditioning on an *intermediate* x_t that already carries structural information, the model can progressively refine its estimate rather than collapsing to the mean.

✓ Flow Matching

So one-step denoising did not work too well, huh? Instead, one could iteratively denoise the image, and we'll do so with flow matching by training a UNet model to predict the *flow* from our noisy data to clean data. In our flow matching setup, we sample a pure noise image $x_1 \sim \mathcal{N}(0, I)$ and denoise into a valid image x_0 .

For iterative denoising, we need to define how intermediate noisy samples are constructed. The simplest approach would be a linear interpolation between noisy x_1 and clean x_0 for some x_0 in our training data, such that

$$x_t = (1 - t)x_0 + tx_1 \quad \text{where } x_1 \sim \mathcal{N}(0, I), t \in [0, 1].$$

Like so, we define a vector field describing the position of a point x_t at time t relative to the clean data distribution and the noisy data distribution. Intuitively, we observe that for large t , x_t remains close to noise, while for $t \rightarrow 0$, we approach the clean distribution. Flow can be thought of as the velocity (change in position with respect to time) of such vector field, describing how to move from x_0 to x_1 .

$$v(x_t, t) = \frac{d}{dt} x_t = x_1 - x_0.$$

So we aim to learn a UNet $v_\theta(x_t, t)$ which approximates this flow $v(x_t, t) = x_1 - x_0$, such that:

$$L = \mathbb{E}_{x_0 \sim p_0(x_0), x_1 \sim p_1(x_1), t \sim U[0,1]} \|(x_1 - x_0) - v_\theta(x_t, t)\|^2.$$

Notice that our **UNet** no longer only takes an image-like input like x_t , but it's also conditioned on t . Before training our flow model we should build on top of our previous **UNet** and **ResBlock** such that they allow for conditioning on t . While there are many ways of doing this we suggest doing as the diagram below.

Proj (a, b) – **nn.Linear(in_features=a, out_features=b)**



```
class ResBlock(nn.Module):
    def __init__(self, in_channels: int, out_channels: int, t_dim: int):
        super().__init__()
        self.norm1 = nn.GroupNorm(32, in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)
        self.t_proj = nn.Linear(t_dim, out_channels)
        self.norm2 = nn.GroupNorm(32, out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1)
        self.skip = (
            nn.Conv2d(in_channels, out_channels, kernel_size=1, padding=0)
            if in_channels != out_channels
            else nn.Identity()
        )

    def forward(self, x: torch.Tensor, t: torch.Tensor):
        h = F.silu(self.norm1(x))
        h = self.conv1(h)
        h = h + self.t_proj(t)[: , :, None, None]
```

```

h = F.silu(self.norm2(h))
h = self.conv2(h)
return h + self.skip(x)

```

Now we update our **UNet** to include our time-conditioned **ResBlock**. Note that we should embed the scalar timestep t using sinusoidal positional embeddings with log-spaced frequencies

$$\omega_k = \exp\left(-k \cdot \frac{\log \theta}{\frac{d}{2} - 1}\right), \quad k = 0, \dots, \frac{d}{2} - 1,$$

and define

$$\text{PE}(t) = \left[\sin(\omega_0 t), \dots, \sin(\omega_{d/2-1} t), \cos(\omega_0 t), \dots, \cos(\omega_{d/2-1} t) \right].$$

This maps $t \in [0, 1]$ into a smooth, fixed-dimensional representation that enables the network to condition on different noise levels.

```

class SinusoidalPositionalEmbeddings(nn.Module):
    def __init__(self, dim: int, theta: float = 10000.0):
        super().__init__()
        assert dim % 2 == 0
        self.dim = dim
        self.theta = theta

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        device = x.device
        half_dim = self.dim // 2
        k = torch.arange(half_dim, device=device, dtype=torch.float32)
        omega = torch.exp(-k * math.log(self.theta) / max(half_dim - 1, 1))
        angles = x[:, None] * omega[None, :]
        return torch.cat([torch.sin(angles), torch.cos(angles)], dim=-1)

class TimeEmbedding(nn.Module):
    def __init__(self, t_dim: int, hidden_dim: int):
        super().__init__()
        self.pos = SinusoidalPositionalEmbeddings(t_dim)
        self.mlp = nn.Sequential(
            nn.Linear(t_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, t_dim)
        )

    def forward(self, t: torch.Tensor):
        t = self.pos(t)
        return self.mlp(t)

```

TimeEmbed (a, b) – `TimeEmbedding(t_dim=a, hidden_dim=b)`

ResBlock (a, b, c) – `ResBlock(in_channels=a, out_channels=b, t_dim=c)`



```
class UNet(nn.Module):
    def __init__(
        self,
        in_channels: int = 1,
        hidden_dim: int = 32,
        t_dim: int = 64
    ):
        super().__init__()
        D = hidden_dim

        self.time_emb = TimeEmbedding(t_dim=t_dim, hidden_dim=4 * t_dim)

        self.init_conv = nn.Conv2d(in_channels, D, kernel_size=3, padding=1)
        self.down1_res = ResBlock(D, D, t_dim)
        self.down1_attn = AttnBlock(D, num_heads=4)

        self.down2_conv = nn.Conv2d(D, D, kernel_size=3, stride=2, padding=1)
        self.down2_res = ResBlock(D, 2 * D, t_dim)
        self.down2_attn = AttnBlock(2 * D, num_heads=4)

        self.down3_conv = nn.Conv2d(2 * D, 2 * D, kernel_size=3, stride=2, padding=1)
        self.down3_res = ResBlock(2 * D, 4 * D, t_dim)

        self.mid_res1 = ResBlock(4 * D, 4 * D, t_dim)
        self.mid_attn = AttnBlock(4 * D, num_heads=4)
        self.mid_res2 = ResBlock(4 * D, 4 * D, t_dim)

        self.up3_res = ResBlock(4 * D + 4 * D, 4 * D, t_dim)
        self.up3_conv = nn.Sequential(
            nn.Upsample(scale_factor=2),
            nn.Conv2d(4 * D, 4 * D, kernel_size=3, padding=1),
        )

        self.up2_res = ResBlock(4 * D + 2 * D, 2 * D, t_dim)
        self.up2_attn = AttnBlock(2 * D, num_heads=4)
        self.up2_conv = nn.Sequential(
            nn.Upsample(scale_factor=2),
            nn.Conv2d(2 * D, 2 * D, kernel_size=3, padding=1),
        )

        self.up1_res = ResBlock(2 * D + D, D, t_dim)
        self.up1_attn = AttnBlock(D, num_heads=4)
```

```

self.out_norm = nn.GroupNorm(32, D)
self.out_conv = nn.Conv2d(D, in_channels, kernel_size=3, padding=1)

def forward(self, x: torch.Tensor, t: torch.Tensor):
    t_emb = self.time_emb(t)
    h = self.init_conv(x)

    h = self.down1_res(h, t_emb)
    skip1 = self.down1_attn(h)

    h = self.down2_conv(skip1)
    h = self.down2_res(h, t_emb)
    skip2 = self.down2_attn(h)

    h = self.down3_conv(skip2)
    skip3 = self.down3_res(h, t_emb)

    h = self.mid_res1(skip3, t_emb)
    h = self.mid_attn(h)
    h = self.mid_res2(h, t_emb)

    h = torch.cat([h, skip3], dim=1)
    h = self.up3_res(h, t_emb)
    h = self.up3_conv(h)

    h = torch.cat([h, skip2], dim=1)
    h = self.up2_res(h, t_emb)
    h = self.up2_attn(h)
    h = self.up2_conv(h)

    h = torch.cat([h, skip1], dim=1)
    h = self.up1_res(h, t_emb)
    h = self.up1_attn(h)

    h = F.silu(self.out_norm(h))
    return self.out_conv(h)

```

Now you should implement the following

`FlowMatching.loss` – (1) takes a clean image x_0 , (2) samples noise and a time t , (3) produces $x_t = (1 - t)x_0 + tx_1$ via interpolation and target velocity $v = x_1 - x_0$. and (4) returns the mean squared error (MSE) loss between the model's predicted velocity and the target velocity.

`FlowMatching.sample` – (1) initializes x_t as noise or x_1 if provided, (2) denoises x_t iteratively for `num_inference_steps` T via $x_t \leftarrow x_t - \frac{1}{T}\hat{v}$ and (3) returns the final denoised image.

```

class FlowMatching(nn.Module):
    def __init__(self, model: nn.Module):
        super().__init__()
        self.model = model

    def loss(self, x_0: torch.Tensor) -> torch.Tensor:
        B = x_0.shape[0]
        x_1 = torch.randn_like(x_0)
        t = torch.rand(B, device=x_0.device)
        t4 = t[:, None, None, None]
        x_t = (1 - t4) * x_0 + t4 * x_1
        v_target = x_1 - x_0
        v_pred = self.model(x_t, t)
        return F.mse_loss(v_pred, v_target)

    @torch.no_grad()
    def sample(
        self,
        size: torch.Size | None = None,
        x_1: torch.Tensor | None = None,
        num_inference_steps: int = 50,
        device: torch.device = None,
    ) -> torch.Tensor:
        if x_1 is not None:
            x = x_1.clone().to(device)
        else:
            x = torch.randn(size, device=device)

        for i in tqdm(range(num_inference_steps), leave=False):
            t_val = 1.0 - i / num_inference_steps
            t = torch.full((x.shape[0],), t_val, device=x.device)
            v = self.model(x, t)
            x = x - (1.0 / num_inference_steps) * v

        return x

```

Train the time-conditioned flow matching model.

- Use `hidden_dim=32` and `t_dim=64`, and train for 10 epochs.
- Use `torch.optim.Adam` with learning rate 10^{-3} .

Deliverables:

- Plot the training loss curve over all iterations.
- Show sampling results after epochs 5 and 10.

```
NUM_EPOCHS_FLOW = 10
```

```

unet_flow = UNet(in_channels=C, hidden_dim=32, t_dim=64).to(device)
flow_model = FlowMatching(unet_flow).to(device)

```

```
optimizer_flow = torch.optim.Adam(flow_model.parameters(), lr=1e-3)
flow_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)

flow_losses = []
flow_snapshots = {}

for epoch in range(1, NUM_EPOCHS_FLOW + 1):
    flow_model.train()
    for x, _ in tqdm(flow_loader, desc=f'Flow epoch {epoch}/{NUM_EPOCHS_FLOW}'):
        x = x.to(device)
        loss = flow_model.loss(x)
        optimizer_flow.zero_grad()
        loss.backward()
        optimizer_flow.step()
        flow_losses.append(loss.item())

    if epoch in (1, 5, 10):
        flow_model.eval()
        samples = flow_model.sample(
            size=torch.Size([16, C, H, W]),
            num_inference_steps=50,
            device=device,
        )
        flow_snapshots[epoch] = samples.cpu()

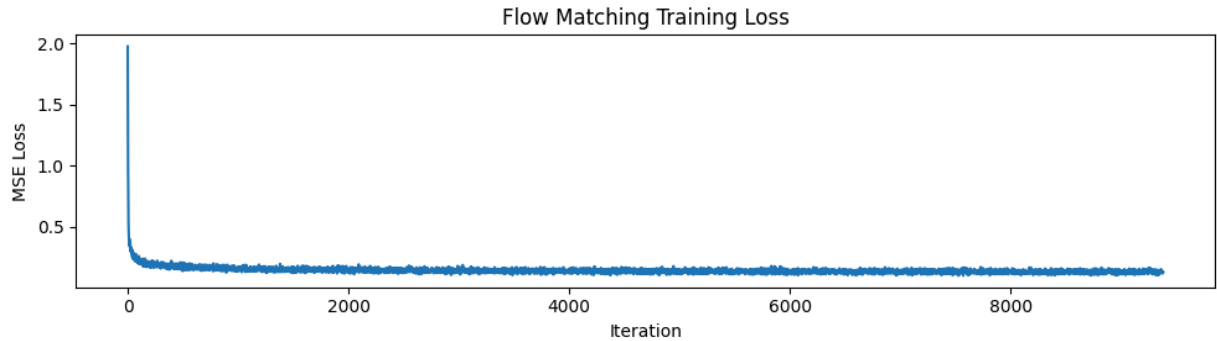
torch.save(flow_model.state_dict(), 'checkpoints/flow_matching.pt')

plt.figure(figsize=(10, 3))
plt.plot(flow_losses)
plt.xlabel('Iteration')
plt.ylabel('MSE Loss')
plt.title('Flow Matching Training Loss')
plt.tight_layout()
plt.savefig('figures/flow_loss.png', dpi=100)
plt.show()
```

```

Flow epoch 1/10: 100%|██████████| 938/938 [00:33<00:00, 27.95it/s]
Flow epoch 2/10: 100%|██████████| 938/938 [00:33<00:00, 28.14it/s]
Flow epoch 3/10: 100%|██████████| 938/938 [00:33<00:00, 28.12it/s]
Flow epoch 4/10: 100%|██████████| 938/938 [00:33<00:00, 28.14it/s]
Flow epoch 5/10: 100%|██████████| 938/938 [00:33<00:00, 28.17it/s]
Flow epoch 6/10: 100%|██████████| 938/938 [00:33<00:00, 28.14it/s]
Flow epoch 7/10: 100%|██████████| 938/938 [00:33<00:00, 28.14it/s]
Flow epoch 8/10: 100%|██████████| 938/938 [00:33<00:00, 28.15it/s]
Flow epoch 9/10: 100%|██████████| 938/938 [00:33<00:00, 28.15it/s]
Flow epoch 10/10: 100%|██████████| 938/938 [00:33<00:00, 28.15it/s]

```

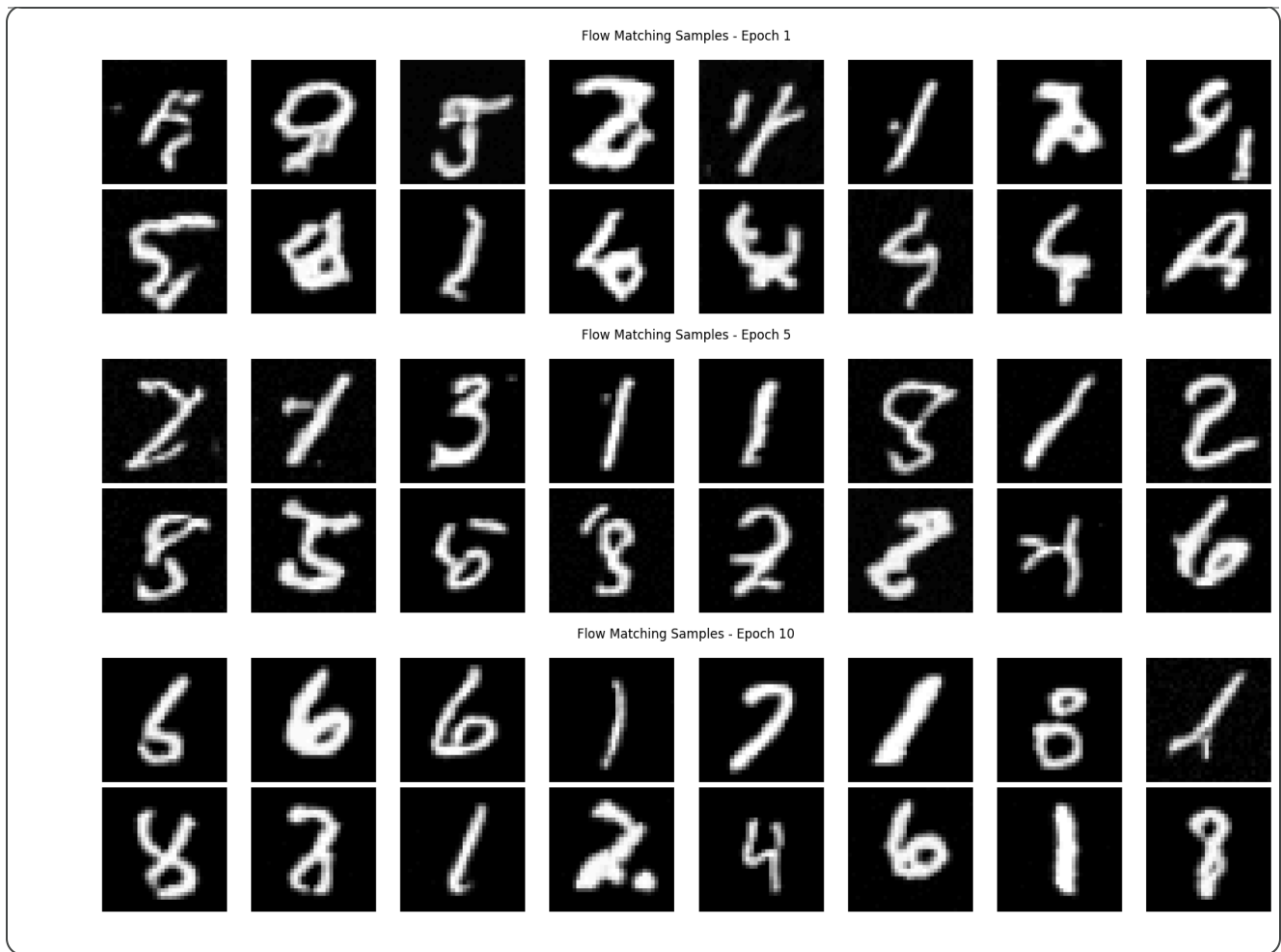


Visualize generated samples from your trained flow matching model. Display results after epochs 5 and 10.

```

for epoch, samples in sorted(flow_snapshots.items()):
    n_cols = 8
    n_rows = len(samples) // n_cols
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(2 * n_cols, 2 * n_rows))
    for idx, ax in enumerate(axes.flatten()):
        ax.imshow(denormalize(samples[idx]).squeeze().clamp(0, 1), cmap='gray')
        ax.axis('off')
    plt.suptitle(f'Flow Matching Samples - Epoch {epoch}')
    plt.tight_layout()
    plt.savefig(f'figures/flow_samples_epoch{epoch}.png', dpi=100)
    plt.show()

```



✓ Class-Conditioned Flow Matching

By now, you should have a not-so-effective, but working, ten-sided die. How could we condition it so we could choose which number to generate?

There are numerous ways one could go about it. We could modify our `ResBlock` to be conditioned on both time and class as the diagram below depicts.



```
class ResBlock(nn.Module):
    def __init__(
        self, in_channels: int, out_channels: int, t_dim: int, cls_dim: int | None
    ):
        super().__init__()
        self.norm1 = nn.GroupNorm(32, in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)
        self.t_proj = nn.Linear(t_dim, out_channels)
        self.cls_proj = nn.Linear(cls_dim, out_channels) if cls_dim is not None else None
        self.norm2 = nn.GroupNorm(32, out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1)
```

```

        self.skip = (
            nn.Conv2d(in_channels, out_channels, kernel_size=1, padding=0)
            if in_channels != out_channels
            else nn.Identity()
        )

    def forward(
        self, x: torch.Tensor, t: torch.Tensor, cls: torch.Tensor | None = None
    ):
        h = F.silu(self.norm1(x))
        h = self.conv1(h)
        h = h + self.t_proj(t)[:, :, None, None]
        if cls is not None and self.cls_proj is not None:
            h = h + self.cls_proj(cls)[:, :, None, None]
        h = F.silu(self.norm2(h))
        h = self.conv2(h)
        return h + self.skip(x)

```

ClsEmbed (a, b) – `nn.Embedding(num_embeddings=a, embedding_dim=b)`

ResBlock (a, b, c, d) – `ResBlock(in_channels=a, out_channels=b, t_dim=c, cls_dim=d)`



```

class UNet(nn.Module):
    def __init__(
        self,
        in_channels: int = 1,
        hidden_dim: int = 32,
        t_dim: int = 64,
        cls_dim: int | None = None,
        num_classes: int | None = 10,
    ):
        super().__init__()
        D = hidden_dim

        self.time_emb = TimeEmbedding(t_dim=t_dim, hidden_dim=4 * t_dim)

        self.cls_emb = (
            nn.Embedding(num_classes + 1, cls_dim)
            if (cls_dim is not None and num_classes is not None)
            else None
        )
        self.num_classes = num_classes

        self.init_conv = nn.Conv2d(in_channels, D, kernel_size=3, padding=1)

        self.down1_res = ResBlock(D, D, t_dim, cls_dim)

```

```

self.down1_attn = AttnBlock(D, num_heads=4)

self.down2_conv = nn.Conv2d(D, D, kernel_size=3, stride=2, padding=1)
self.down2_res = ResBlock(D, 2 * D, t_dim, cls_dim)
self.down2_attn = AttnBlock(2 * D, num_heads=4)

self.down3_conv = nn.Conv2d(2 * D, 2 * D, kernel_size=3, stride=2, padding=1)
self.down3_res = ResBlock(2 * D, 4 * D, t_dim, cls_dim)

self.mid_res1 = ResBlock(4 * D, 4 * D, t_dim, cls_dim)
self.mid_attn = AttnBlock(4 * D, num_heads=4)
self.mid_res2 = ResBlock(4 * D, 4 * D, t_dim, cls_dim)

self.up3_res = ResBlock(4 * D + 4 * D, 4 * D, t_dim, cls_dim)
self.up3_conv = nn.Sequential(
    nn.Upsample(scale_factor=2),
    nn.Conv2d(4 * D, 4 * D, kernel_size=3, padding=1),
)

self.up2_res = ResBlock(4 * D + 2 * D, 2 * D, t_dim, cls_dim)
self.up2_attn = AttnBlock(2 * D, num_heads=4)
self.up2_conv = nn.Sequential(
    nn.Upsample(scale_factor=2),
    nn.Conv2d(2 * D, 2 * D, kernel_size=3, padding=1),
)

self.up1_res = ResBlock(2 * D + D, D, t_dim, cls_dim)
self.up1_attn = AttnBlock(D, num_heads=4)

self.out_norm = nn.GroupNorm(32, D)
self.out_conv = nn.Conv2d(D, in_channels, kernel_size=3, padding=1)

def forward(
    self,
    x: torch.Tensor,
    t: torch.Tensor,
    cls: torch.Tensor | None = None,
):
    t_emb = self.time_emb(t)
    cls_emb = self.cls_emb(cls) if (cls is not None and self.cls_emb is not None) else None

    h = self.init_conv(x)

    h = self.down1_res(h, t_emb, cls_emb)
    skip1 = self.down1_attn(h)

    h = self.down2_conv(skip1)
    h = self.down2_res(h, t_emb, cls_emb)
    skip2 = self.down2_attn(h)

    h = self.down3_conv(skip2)

```

```

skip3 = self.down3_res(h, t_emb, cls_emb)

h = self.mid_res1(skip3, t_emb, cls_emb)
h = self.mid_attn(h)
h = self.mid_res2(h, t_emb, cls_emb)

h = torch.cat([h, skip3], dim=1)
h = self.up3_res(h, t_emb, cls_emb)
h = self.up3_conv(h)

h = torch.cat([h, skip2], dim=1)
h = self.up2_res(h, t_emb, cls_emb)
h = self.up2_attn(h)
h = self.up2_conv(h)

h = torch.cat([h, skip1], dim=1)
h = self.up1_res(h, t_emb, cls_emb)
h = self.up1_attn(h)

h = F.silu(self.out_norm(h))
return self.out_conv(h)

```

To enable classifier-free guidance (CFG), during training we randomly drop the class conditioning with probability $p_{\text{uncond}} = 0.1$ such that we input a null conditioning signal. At sampling time, we compute both a conditional and unconditional velocity estimate and blend them:

$$\hat{v} = v_{\theta}(x_t, t, \emptyset) + \gamma (v_{\theta}(x_t, t, c) - v_{\theta}(x_t, t, \emptyset))$$

where γ is the guidance scale. You should extend your `FlowMatching` class (or write a new one) to support class-conditioned training with unconditional dropout, and CFG-guided sampling.

```

class FlowMatchingCFG(nn.Module):
    def __init__(self, model: nn.Module, num_classes: int = 10, p_uncond: float):
        super().__init__()
        self.model = model
        self.num_classes = num_classes
        self.p_uncond = p_uncond

    def loss(self, x_0: torch.Tensor, cls: torch.Tensor) -> torch.Tensor:
        B = x_0.shape[0]
        x_1 = torch.randn_like(x_0)
        t = torch.rand(B, device=x_0.device)
        t4 = t[:, None, None, None]
        x_t = (1 - t4) * x_0 + t4 * x_1
        v_target = x_1 - x_0

```

```

        drop_mask = torch.rand(B, device=x_0.device) < self.p_uncond
        cls_in = cls.clone()
        cls_in[drop_mask] = self.num_classes

        v_pred = self.model(x_t, t, cls_in)
        return F.mse_loss(v_pred, v_target)

@torch.no_grad()
def sample(
    self,
    cls: torch.Tensor,
    gamma: float = 5.0,
    num_inference_steps: int = 50,
    device: torch.device = None,
) -> torch.Tensor:
    B = cls.shape[0]
    x = torch.randn(B, C, H, W, device=device)
    cls = cls.to(device)
    null_cls = torch.full((B,), self.num_classes, dtype=torch.long, device=

    for i in tqdm(range(num_inference_steps), leave=False):
        t_val = 1.0 - i / num_inference_steps
        t = torch.full((B,), t_val, device=device)
        v_uncond = self.model(x, t, null_cls)
        v_cond = self.model(x, t, cls)
        v = v_uncond + gamma * (v_cond - v_uncond)
        x = x - (1.0 / num_inference_steps) * v

    return x

```

Train the class-conditioned flow matching model.

- Use `hidden_dim=32`, `t_dim=64`, `cls_dim=64` and train for 10 epochs.
- Use `torch.optim.Adam` with learning rate 10^{-3} .
- During training, drop the class label with probability $p_{\text{uncond}} = 0.1$.

Deliverables:

- Plot the training loss curve.
- Show sampling results after epochs 5 and 10.

```

NUM_CLASSES = 10
NUM_EPOCHS = 10

UNET_CHANNELS = 32
HIDDEN_DIM = 32
T_DIM = 64
CLS_DIM = 64

UNET_CLASSES = 10

UNET_CHANNELS = 32
HIDDEN_DIM = 32
T_DIM = 64
CLS_DIM = 64

UNET_CLASSES = 10

```

```

    num_classes=NUM_CLASSES,
).to(device)

flow_cls = FlowMatchingCFG(unet_cls, num_classes=NUM_CLASSES, p_uncond=0.1).to(
optimizer_cls = torch.optim.Adam(flow_cls.parameters(), lr=1e-3)
cls_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)

cls_losses = []
cls_snapshots = {}

for epoch in range(1, NUM_EPOCHS + 1):
    flow_cls.train()
    for x, y in tqdm(cls_loader, desc=f'Cls epoch {epoch}/{NUM_EPOCHS}'):
        x, y = x.to(device), y.to(device)
        loss = flow_cls.loss(x, y)
        optimizer_cls.zero_grad()
        loss.backward()
        optimizer_cls.step()
        cls_losses.append(loss.item())

    if epoch in (1, 5, 10):
        flow_cls.eval()
        all_samples = []
        for c in range(NUM_CLASSES):
            cls_tensor = torch.full((4,), c, dtype=torch.long)
            samples = flow_cls.sample(
                cls=cls_tensor,
                gamma=5.0,
                num_inference_steps=50,
                device=device,
            )
            all_samples.append(samples.cpu())
        cls_snapshots[epoch] = torch.cat(all_samples, dim=0) # (40, C, H, W)

torch.save(flow_cls.state_dict(), 'checkpoints/flow_cls.pt')

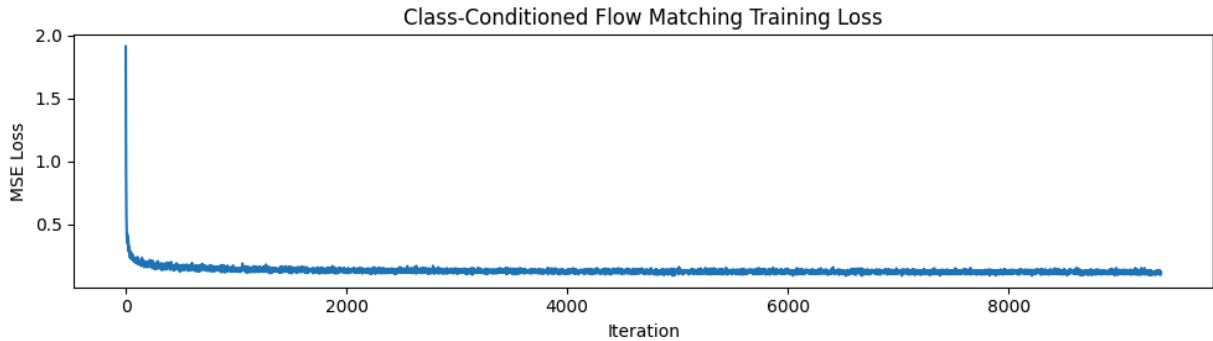
plt.figure(figsize=(10, 3))
plt.plot(cls_losses)
plt.xlabel('Iteration')
plt.ylabel('MSE Loss')
plt.title('Class-Conditioned Flow Matching Training Loss')
plt.tight_layout()
plt.savefig('figures/cls_flow_loss.png', dpi=100)
plt.show()

```

```

Cls epoch 1/10: 100%|██████████| 938/938 [00:33<00:00, 27.79it/s]
Cls epoch 2/10: 100%|██████████| 938/938 [00:33<00:00, 27.79it/s]
Cls epoch 3/10: 100%|██████████| 938/938 [00:33<00:00, 27.75it/s]
Cls epoch 4/10: 100%|██████████| 938/938 [00:33<00:00, 27.76it/s]
Cls epoch 5/10: 100%|██████████| 938/938 [00:33<00:00, 27.76it/s]
Cls epoch 6/10: 100%|██████████| 938/938 [00:33<00:00, 27.75it/s]
Cls epoch 7/10: 100%|██████████| 938/938 [00:33<00:00, 27.76it/s]
Cls epoch 8/10: 100%|██████████| 938/938 [00:33<00:00, 27.79it/s]
Cls epoch 9/10: 100%|██████████| 938/938 [00:33<00:00, 27.77it/s]
Cls epoch 10/10: 100%|██████████| 938/938 [00:33<00:00, 27.79it/s]

```



Sample from the class-conditioned model using classifier-free guidance with $\gamma = 5.0$. Generate 4 images per digit class (0–9) and display them in a grid.

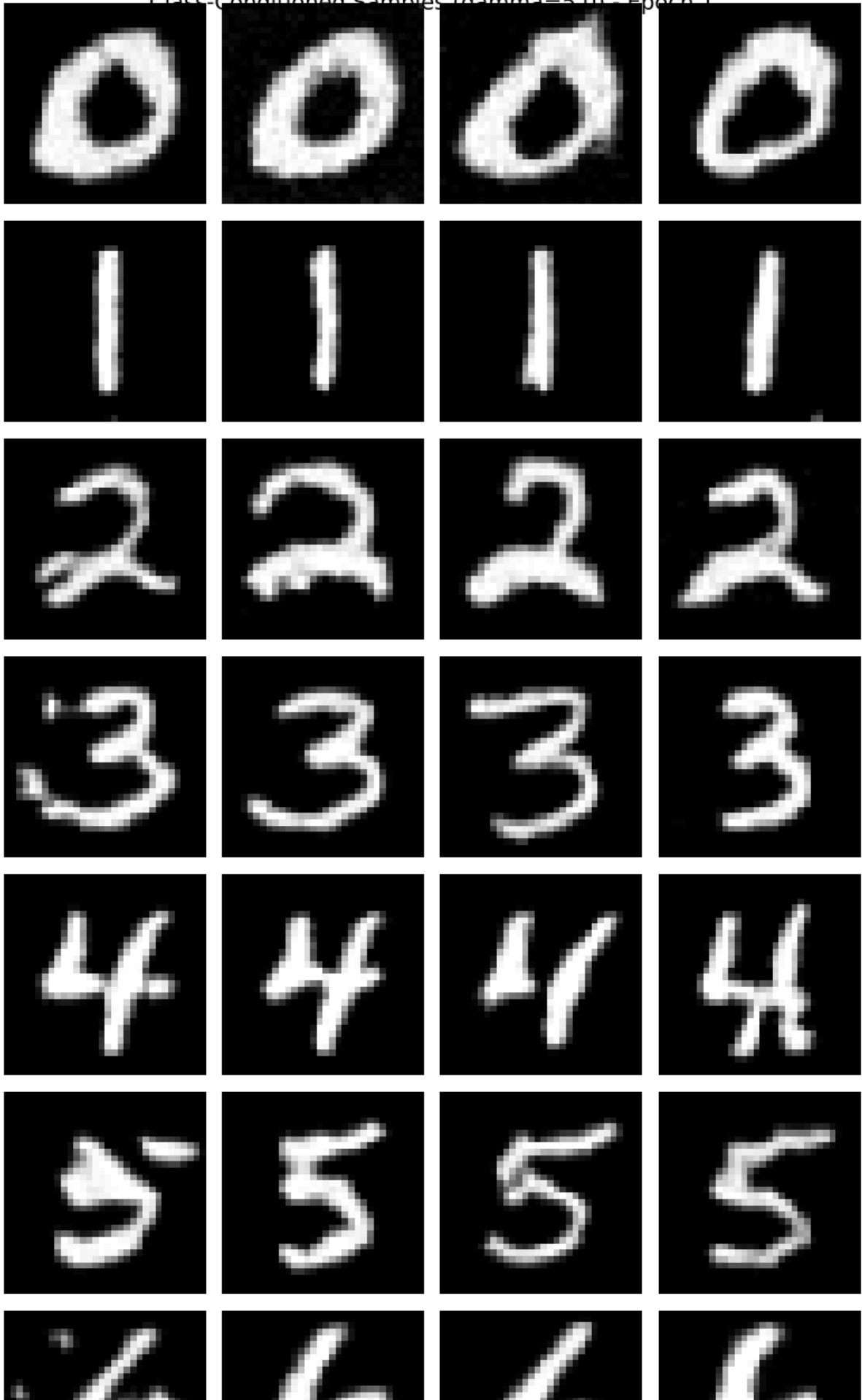
Deliverables:

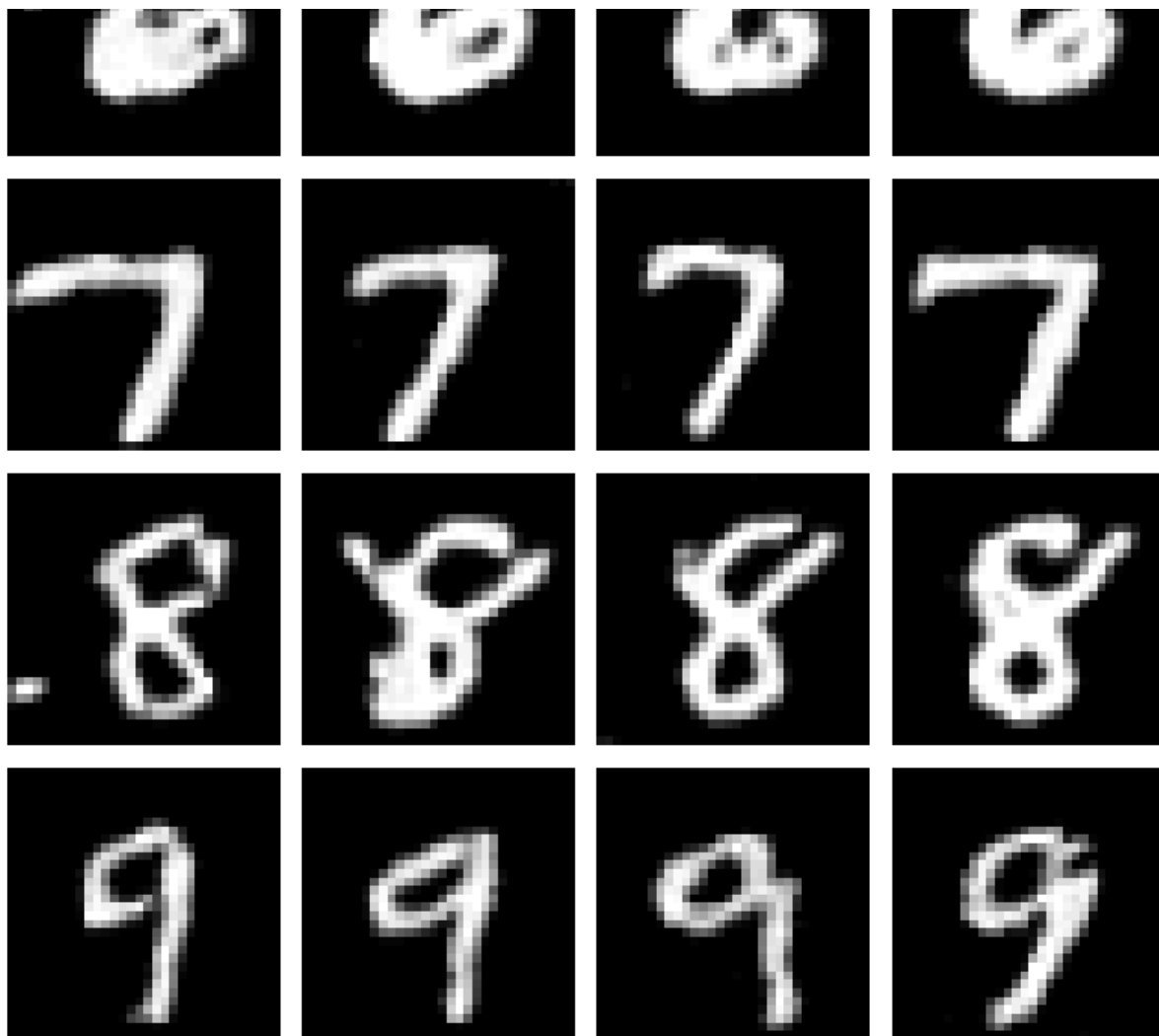
- A 10×4 grid of class-conditioned samples after epochs 5 and 10.

```

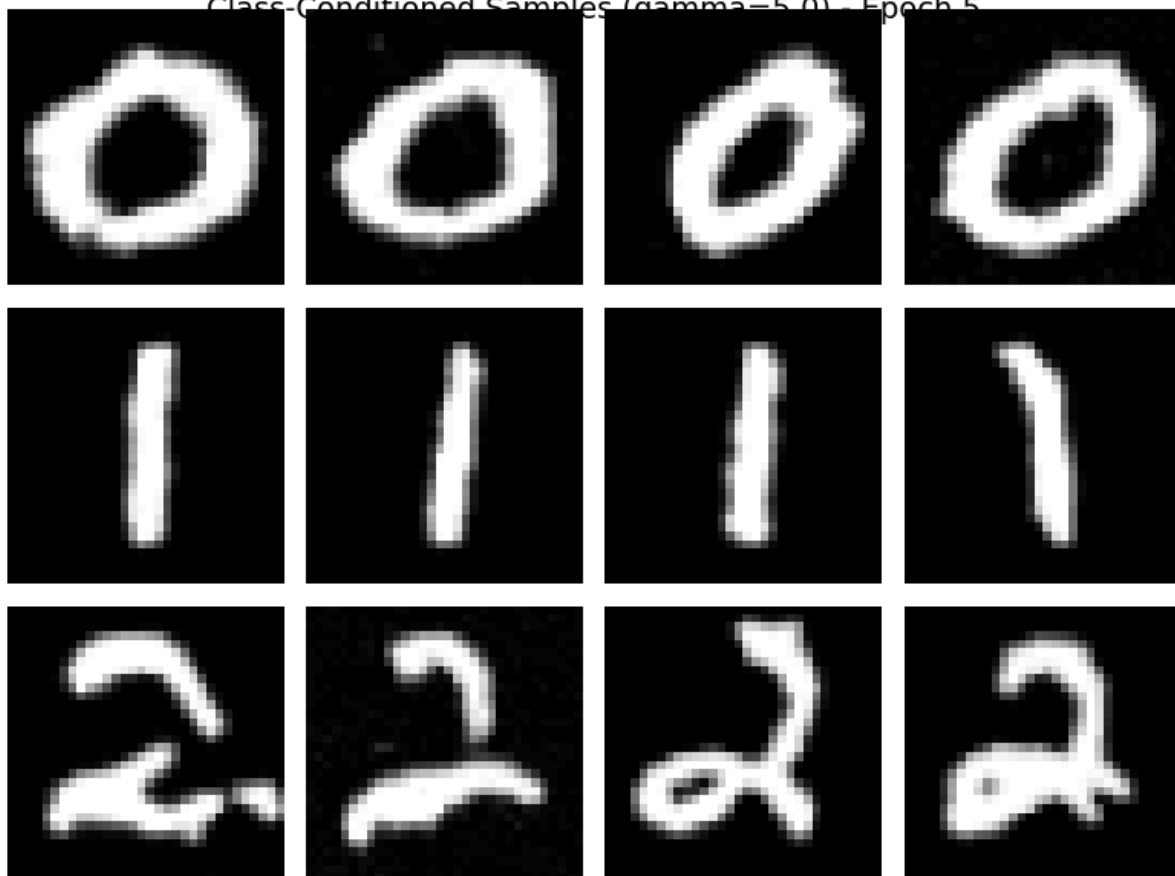
for epoch, samples in sorted(cls_snapshots.items()):
    fig, axes = plt.subplots(10, 4, figsize=(8, 20))
    for cls_idx in range(10):
        for sample_idx in range(4):
            img = denormalize(samples[cls_idx * 4 + sample_idx]).squeeze().clamp(
                -1, 1)
            axes[cls_idx, sample_idx].imshow(img, cmap='gray')
            axes[cls_idx, sample_idx].axis('off')
        axes[cls_idx, 0].set_ylabel(str(cls_idx), fontsize=12)
    plt.suptitle(f'Class-Conditioned Samples (gamma=5.0) - Epoch {epoch}', font
    plt.tight_layout()
    plt.savefig(f'figures/cls_samples_epoch{epoch}.png', dpi=100)
    plt.show()

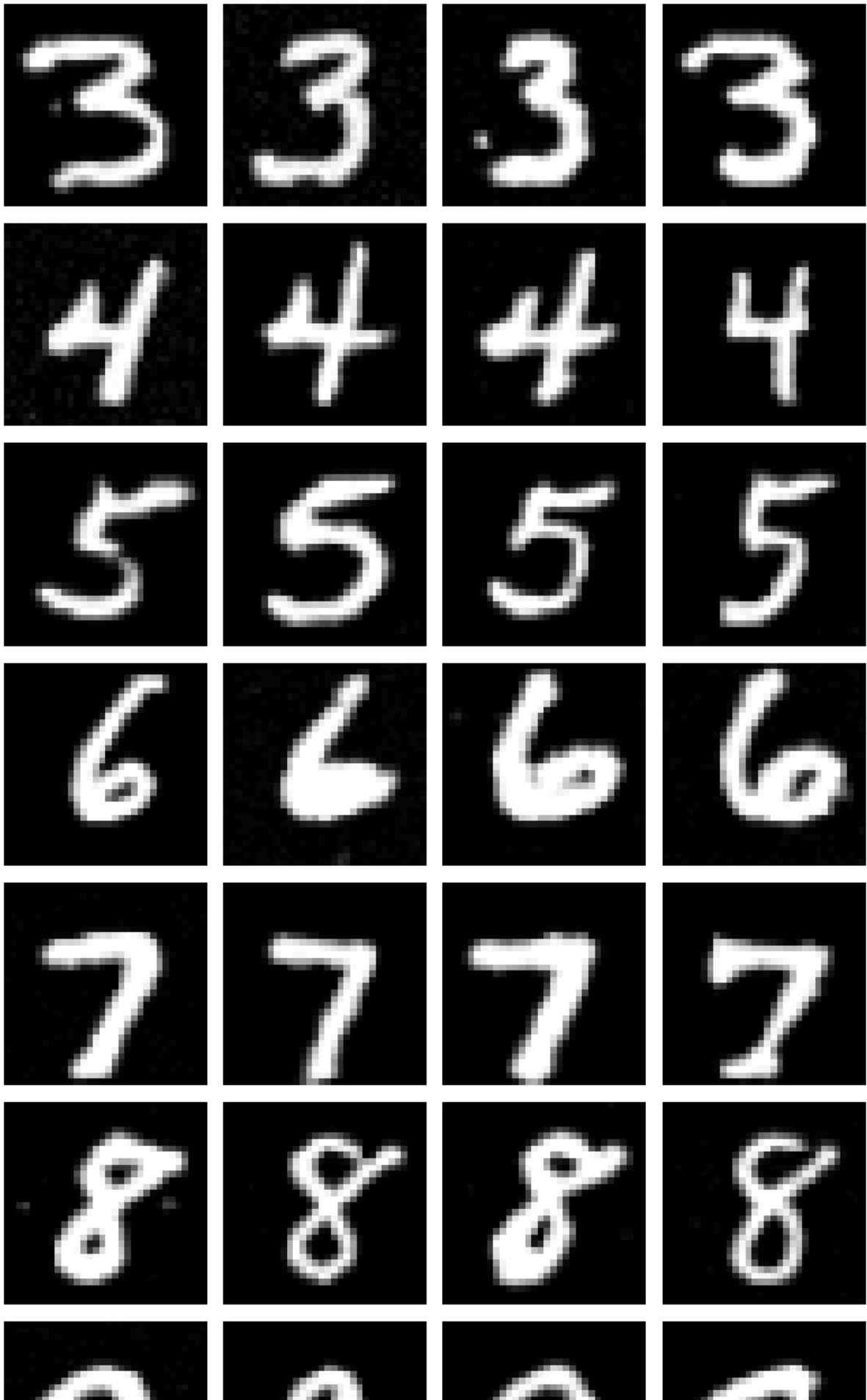
```


Class-Conditioned Samples ($\gamma=5.0$) - Epoch 1



Class-Conditioned Samples (gamma=5.0) - Epoch 5





CS 280: Ideal Flow Machines

You'll be implementing your own ideal flow machines. This project was developed by [Jorge Diaz Chao](#), [Jiaxin Ge](#), [Angjoo Kanazawa](#) and [Alexei A. Efros](#).

```
import torch
import torchvision
import numpy as np
from tqdm import tqdm
import torch.nn.functional as F
import matplotlib.pyplot as plt
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Subset
```

Introduction

How do diffusion and/or flow models produce novel outputs? Why wouldn't they just reproduce the training dataset? Where's the creativity coming from? Researches are actively working on these questions. Throughout this project we'll be looking at a paper with very interesting insights.

In *An Analytic Theory of Creativity in Convolutional Diffusion Models* ([arXiv:2412.20292](#)), Mason Kamb and Surya Ganguli develop analytical models that approximate the ideal diffusion score and achieve impressive results.

You'll be implementing such models in the context of flow matching and comparing them to the outputs of your trained UNet on MNIST.

Note that before proceeding, you should read the paper.

Starter Code

This code is provided as a starting point only. Every component can be freely modified, replaced, or deleted as needed.

```
N_DATA = 10_000
N_STEPS = 20
BATCH_SIZE = 64
C, H, W = 1, 32, 32
LABEL = None
N_SAMPLES = 1
SEED = 1
```

```
if torch.cuda.is_available():
    device = torch.device('cuda')
elif torch.backends.mps.is_available():
    device = torch.device('mps')
else:
    device = torch.device('cpu')
print(f"Using device: {device}")
```

Using device: cuda

Loading the Dataset

We'll be working with MNIST, although you could get experimental and try any other dataset. The original paper shows results on CIFAR-10, FashionMNIST and Celeb-A too.

```
def load_mnist(n_samples):
    transform = transforms.Compose([
        transforms.Resize((H, W)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.5], std=[0.5]),
    ])
    full = torchvision.datasets.MNIST(
        root='./data', train=True, download=True, transform=transform
    )
    return Subset(full, range(n_samples))

def denormalize(x):
    return x * 0.5 + 0.5

dataset = load_mnist(N_DATA)
dataloader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=False)

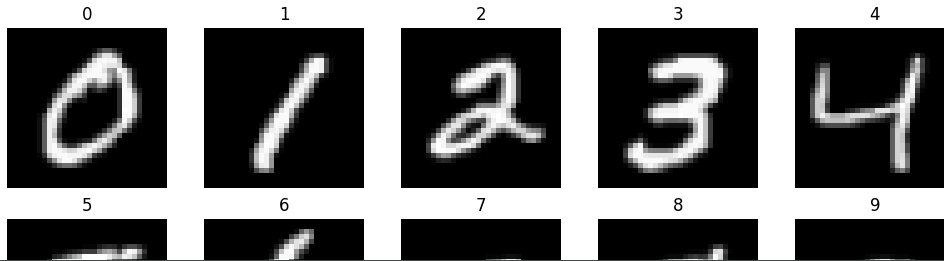
examples = {}
for x, y in dataset:
    if y not in examples:
        examples[y] = x
    if len(examples) == 10:
        break

fig, axes = plt.subplots(2, 5, figsize=(10, 4))

for digit, ax in zip(range(10), axes.flatten()):
    x = denormalize(examples[digit]).squeeze(0)
    ax.imshow(x, cmap="gray")
    ax.set_title(f"{digit}")
    ax.axis("off")

plt.tight_layout()
plt.show()
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 37.0MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 957kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 8.72MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 33.7MB/s]
```



✓ Your UNet as Reference

Paste your UNet implementation here (or import it) and load a saved checkpoint. Ensure your sampling code can start from a provided noise initialization x_1 so generations are reproducible and comparable for different methods.

```
import math
import torch.nn as nn

class SinusoidalPositionalEmbeddings(nn.Module):
    def __init__(self, dim: int, theta: float = 10000.0):
        super().__init__()
        assert dim % 2 == 0
        self.dim = dim
        self.theta = theta

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        device = x.device
        half_dim = self.dim // 2
        k = torch.arange(half_dim, device=device, dtype=torch.float32)
        omega = torch.exp(-k * math.log(self.theta) / max(half_dim - 1, 1))
        angles = x[:, None] * omega[None, :]
        return torch.cat([torch.sin(angles), torch.cos(angles)], dim=-1)

class TimeEmbedding(nn.Module):
    def __init__(self, t_dim: int, hidden_dim: int):
        super().__init__()
        self.pos = SinusoidalPositionalEmbeddings(t_dim)
        self.mlp = nn.Sequential(
            nn.Linear(t_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, t_dim)
        )

    def forward(self, t: torch.Tensor) -> torch.Tensor:
        return self.mlp(self.pos(t))

class AttnBlock(nn.Module):
    def __init__(self, channels: int, num_heads: int = 4):
        super().__init__()
        self.norm = nn.GroupNorm(8, channels)
        self.attn = nn.MultiheadAttention(embed_dim=channels, num_heads=num_heads, batch_first=True)

    def forward(self, x: torch.Tensor):
        B, C, H, W = x.shape
        h = self.norm(x)
        h = h.view(B, C, H * W).permute(0, 2, 1)
        h, _ = self.attn(h, h, h)
        h = h.permute(0, 2, 1).view(B, C, H, W)
        return x + h

class ResBlock(nn.Module):
    def __init__(self, in_channels, out_channels, t_dim, cls_dim=None):
        super().__init__()
        self.norm1 = nn.GroupNorm(8, in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)
        self.t_proj = nn.Linear(t_dim, out_channels)
        self.cls_proj = nn.Linear(cls_dim, out_channels) if cls_dim is not None else None
        self.norm2 = nn.GroupNorm(8, out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1)
        self.skip = (
            nn.Conv2d(in_channels, out_channels, kernel_size=1, padding=0)
            if in_channels != out_channels
            else nn.Identity()
        )

    def forward(self, x, t, cls=None):
        h = F.silu(self.norm1(x))
        h = self.conv1(h)
        h = h + self.t_proj(t)[ :, :, None, None]
        if cls is not None and self.cls_proj is not None:
            h = h + self.cls_proj(cls)[ :, :, None, None]
        h = F.silu(self.norm2(h))
        h = self.conv2(h)
        return h + self.skip(x)

class UNet(nn.Module):
    def __init__(self, in_channels=1, hidden_dim=32, t_dim=64, cls_dim=None, num_classes=10):
```

```

    super().__init__()
    D = hidden_dim
    self.time_emb = TimeEmbedding(t_dim=t_dim, hidden_dim=4 * t_dim)
    self.cls_emb = (
        nn.Embedding(num_classes + 1, cls_dim)
        if (cls_dim is not None and num_classes is not None)
        else None
    )
    self.num_classes = num_classes
    self.init_conv = nn.Conv2d(in_channels, D, kernel_size=3, padding=1)
    self.down1_res = ResBlock(D, D, t_dim, cls_dim)
    self.down1_conv = nn.Conv2d(D, 2 * D, kernel_size=3, stride=2, padding=1)
    self.down2_res = ResBlock(2 * D, 2 * D, t_dim, cls_dim)
    self.down2_attn = AttnBlock(2 * D, num_heads=4)
    self.down2_conv = nn.Conv2d(2 * D, 4 * D, kernel_size=3, stride=2, padding=1)
    self.mid_res1 = ResBlock(4 * D, 4 * D, t_dim, cls_dim)
    self.mid_attn = AttnBlock(4 * D, num_heads=4)
    self.mid_res2 = ResBlock(4 * D, 4 * D, t_dim, cls_dim)
    self.up2_conv = nn.Sequential(nn.Upsample(scale_factor=2), nn.Conv2d(4 * D, 2 * D, kernel_size=3, padding=1))
    self.up2_res = ResBlock(4 * D, 2 * D, t_dim, cls_dim)
    self.up2_attn = AttnBlock(2 * D, num_heads=4)
    self.up1_conv = nn.Sequential(nn.Upsample(scale_factor=2), nn.Conv2d(2 * D, D, kernel_size=3, padding=1))
    self.up1_res = ResBlock(2 * D, D, t_dim, cls_dim)
    self.out_norm = nn.GroupNorm(8, D)
    self.out_conv = nn.Conv2d(D, in_channels, kernel_size=3, padding=1)

def forward(self, x, t, cls=None):
    t_emb = self.time_emb(t)
    cls_emb = self.cls_emb(cls) if (cls is not None and self.cls_emb is not None) else None
    h = self.init_conv(x)
    skip1 = self.down1_res(h, t_emb, cls_emb)
    h = self.down1_conv(skip1)
    h = self.down2_res(h, t_emb, cls_emb)
    skip2 = self.down2_attn(h)
    h = self.down2_conv(skip2)
    h = self.mid_res1(h, t_emb, cls_emb)
    h = self.mid_attn(h)
    h = self.mid_res2(h, t_emb, cls_emb)
    h = self.up2_conv(h)
    h = torch.cat([h, skip2], dim=1)
    h = self.up2_res(h, t_emb, cls_emb)
    h = self.up2_attn(h)
    h = self.up1_conv(h)
    h = torch.cat([h, skip1], dim=1)
    h = self.up1_res(h, t_emb, cls_emb)
    h = F.silu(self.out_norm(h))
    return self.out_conv(h)

class FlowMatching(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model

    @torch.no_grad()
    def sample(self, x_1, num_inference_steps=50, device=None):
        x = x_1.clone().to(device)
        for i in range(num_inference_steps):
            t_val = 1.0 - i / num_inference_steps
            t = torch.full((x.shape[0],), t_val, device=x.device)
            v = self.model(x, t)
            x = x - (1.0 / num_inference_steps) * v
        return x

# Load the trained flow matching model
UNET = UNet(in_channels=1, hidden_dim=32, t_dim=64)
model = FlowMatching(UNET).to(device)
ckpt_path = 'checkpoints/flow_matching.pt'
import os
if os.path.exists(ckpt_path):
    model.load_state_dict(torch.load(ckpt_path, map_location=device))
    print(f"Loaded checkpoint from {ckpt_path}")
else:
    print(f"WARNING: No checkpoint found at {ckpt_path}. Train the model in flow_train.ipynb first.")

```

Loaded checkpoint from checkpoints/flow_matching.pt

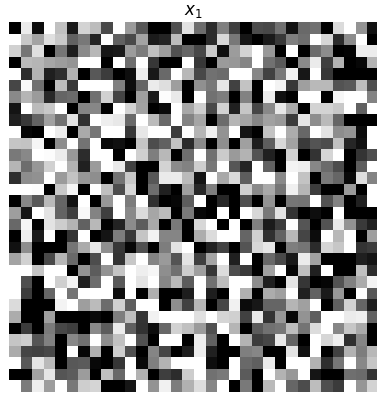
```

import os
if os.path.exists("x_1.pt"):
    x_1 = torch.load("x_1.pt", map_location=device)
    print("Loaded x_1 from x_1.pt")
else:
    torch.manual_seed(SEED)
    x_1 = torch.randn(1, C, H, W, device=device)
    print("Generated fresh x_1 from seed")

x_1_img = denormalize(x_1[0]).cpu().squeeze().clamp(0, 1)
plt.imshow(x_1_img, cmap='gray')
plt.axis('off')
plt.title("$x_1$")
plt.show()

```

Loaded x_1 from x_1.pt

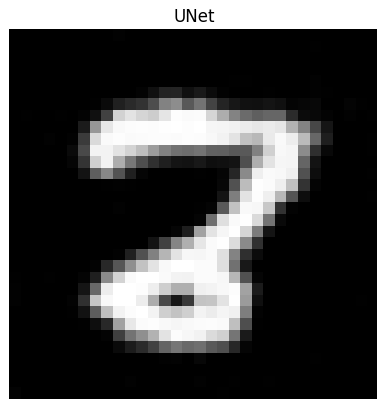


```
print("UNet")

model.eval()
x_unet = model.sample(x_1, num_inference_steps=N_STEPS, device=device)

x_unet_img = denormalize(x_unet[0]).cpu().squeeze().clamp(0, 1)
plt.imshow(x_unet_img, cmap='gray')
plt.axis('off')
plt.title("UNet")
plt.show()
```

UNet



In the exercises below, you will implement **non-trainable analytic models** that approximate the ideal score/flow. You will use them to mimic your trained UNet's sampling dynamics and compare the images they produce to your UNet's outputs.

▼ Euler Sampler

We sample an image by numerically integrating the flow-matching ODE backward from noise just like we did for our trained model. At each step, the vector field is provided by an analytical model such (e.g. ELS / bbELS) that could depend on a per-step patch scale (or effective receptive field) k .

Let $N = \text{len}(\text{scales})$. Initialize

$$x_1 \sim \mathcal{N}(0, I) \quad (\text{or } x_1 = x_1).$$

For steps $i = N - 1, \dots, 1$, define

$$t_i = \frac{i}{N}, \quad k_i = \text{scales}[i],$$

evaluate the vector field

$$v_i = \text{model}(t_i, x_i, \text{dataloader}, k = k_i, \dots),$$

and take a backward Euler step

$$x_{i-1} = x_i + \Delta t v_i, \quad \Delta t = -\frac{1}{N-1}.$$

Return x_0 as the generated sample.

Note that your euler sampler should allow you to display intermediate denoising steps.

```
SCALES = [3, 3, 3, 5, 5, 7, 7, 7, 9, 9, 9, 11, 13, 15, 17, 21, 23, 23, 25, 27]
```

```
@torch.no_grad()
def flow_euler_solver(
    model,
    dataloader,
    scales,
    x_1
):
    num_inference_steps = len(scales)
    x = x_1.clone()

    for i in tqdm(range(num_inference_steps - 1, 0, -1)):
        t = torch.tensor(i / num_inference_steps, device=x.device)
```

```

k = scales[i]
v = model(t, x, dataloader, k=k)
dt = -1.0 / (num_inference_steps - 1)
x = x + dt * v

return x

```

✓ Ideal Flow Machines

The paper *An Analytic Theory of Creativity in Convolutional Diffusion Models* ([arXiv:2412.20292](#)) introduces us to numerous ideal score machines which we'll be implementing in order of appearance in the paper and complexity, but not necessarily accuracy. We refer to our machines as ideal flow machines since we are working with a velocity model instead of a score or noise model like the original paper.



You should look at the paper and understand the method, however, here's a brief overview. For every pixel, we define a score over training patches based on how well their $k \times k$ neighborhoods match the neighborhood of x , defined as

$$\ell(p) = -\frac{\|x_{\text{patch}} - a_t p\|_2^2}{2 b_t^2},$$

where the squared distance expands as

$$\|x_{\text{patch}} - a_t p\|^2 = \|x_{\text{patch}}\|^2 - 2a_t \langle x_{\text{patch}}, p \rangle + a_t^2 \|p\|^2.$$

Each patch proposes a local correction based on its center pixel,

$$v(p) = x - a_t p_{\text{center}}.$$

These proposals are aggregated using a softmax-weighted expectation,

$$E_{\text{diff}} = \frac{\sum_p e^{\ell(p)} v(p)}{\sum_p e^{\ell(p)}}.$$

The returned vector field is obtained from this expected correction as

$$\text{velocity} = \frac{E_{\text{diff}}}{b_t} - \frac{x - E_{\text{diff}}}{a_t}.$$

With linear flow matching, $x_t = (1 - t) x_0 + t x_1$, so $a_t = 1 - t$ and $b_t = t$.

The difference between IS, LS, ELS, and bbELS machines are based on whether we use patches or not (each image is a one patch), and which pair of patches we compare to.



✓ Log-sum-exp accumulation (streaming softmax)

For constructions below, we must compute quantities of the form

$$\frac{\sum_j e^{w_j} v_j}{\sum_j e^{w_j}},$$

where w_j are log-weights and v_j are associated values.

When the index set j is very large (such as for all patches in a dataset), it may be impractical or impossible to materialize all w_j and v_j at once.

The provided helper function implements a **numerically stable, streaming log-sum-exp accumulation** of this weighted average across minibatches. It maintains running accumulators for the numerator and denominator while rescaling them as needed using a per-location maximum, ensuring stability even when the log-weights have large magnitude.

Concretely, given minibatches $\{w_m, v_m\}$, it updates running estimates proportional to

$$\text{num} \leftarrow \sum_m e^{w_m} v_m, \quad \text{den} \leftarrow \sum_m e^{w_m},$$

without ever forming the full softmax explicitly.

You may find this useful whenever a softmax-weighted expectation must be computed over a large or streaming set of terms.

```

def lse_accumulate(numerator, denominator, subtraction, exp_args, values):
    if subtraction is None:
        subtraction = torch.amax(exp_args, dim=(0, 1), keepdim=True)
    else:
        new_sub = torch.amax(exp_args, dim=(0, 1), keepdim=True)
        delta = torch.maximum(new_sub, subtraction)
        numerator /= torch.exp(delta - subtraction)
        denominator /= torch.exp(delta - subtraction)[: , 0, :, :]
        subtraction = delta

    exp_vals = torch.exp(exp_args - subtraction)
    numerator += torch.mean(exp_vals[:, :, None, :, :] * values, dim=1)
    denominator += torch.mean(exp_vals, dim=1)

    return numerator, denominator, subtraction

```

Alternatively, if constrained by computational budget or would like to avoid accumulation, compute the same softmax-weighted average over only the K nearest neighbors which would result in an approximation.

IS

The Ideal Score (IS) machine only reproduces the training set. Every denoising step, it moves closer to a weighted average of the training images. It would mimic the behavior of the diffusion and/or flow model if it only were able to memorize the data distribution, which we know to be not true.

```

@torch.no_grad()
def is_vector_field(t, x, dataloader, k):
    b, c, h, w = x.shape
    at = (1 - t).to(x.device)

```

```

bt = t.to(x.device)

numerator = torch.zeros_like(x)
denominator = torch.zeros(b, 1, 1, 1, device=x.device)
max_logw = torch.full((b, 1, 1, 1), -float('inf'), device=x.device)

for images, labels in dataloader:
    images = images.to(x.device)
    M = images.shape[0]

    diff = x[:, None] - at * images[None]
    log_weights = -torch.sum(diff ** 2, dim=(2, 3, 4)) / (2 * bt ** 2)

    batch_max = log_weights.max(dim=1, keepdim=True).values
    new_max = torch.maximum(max_logw.view(b, 1), batch_max)

    denominator = denominator * torch.exp((max_logw - new_max.view(b, 1, 1, 1)))
    numerator = numerator * torch.exp((max_logw - new_max.view(b, 1, 1, 1)))
    max_logw = new_max.view(b, 1, 1, 1)

    weights = torch.exp(log_weights - max_logw.view(b, 1))

    numerator += (weights[:, :, None, None, None] * diff).sum(dim=1)
    denominator += weights.sum(dim=1).view(b, 1, 1, 1)

E_diff = numerator / denominator

vel = E_diff / bt - (x - E_diff) / at
return vel

```

```

print("IS")

x_is = flow_euler_solver(
    model=is_vector_field,
    dataloader=dataloader,
    scales=SCALES,
    x_1=x_1
)

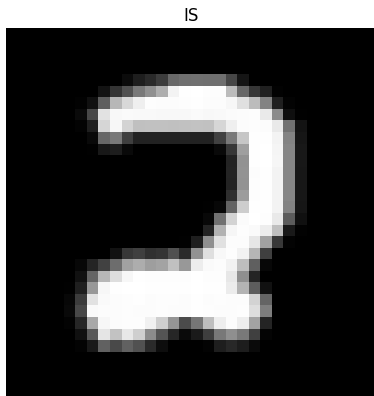
x_is_img = denormalize(x_is[0]).cpu().squeeze().clamp(0, 1)
plt.imshow(x_is_img, cmap='gray')
plt.axis('off')
plt.title("IS")
plt.show()

```

```

IS
100%|██████████| 19/19 [00:09<00:00, 2.03it/s]

```



LS

The Local Score (LS) machine reflects the locality bias of convolutions in the UNet. At each pixel, it compares co-located patches from the training set and moves the pixel toward a weighted average of their center values.

At each pixel location, the model compares its neighborhood patch to training patches from the same location. Implement a function to extract these co-located patches from a batch of images and use it when building the LS machine.

It could be useful for the future if the function returned the squared norm of each patch and its center pixel value, to determine the similarity score and contribution of the center pixel.

```

@torch.no_grad()
def extract_patches_grid(images, k, pad_mode="circular", pad_value=0.0):
    M, C_img, H_img, W_img = images.shape
    pad = k // 2

    if pad_mode == "circular":
        padded = F.pad(images, [pad, pad, pad, pad], mode='circular')
    else:
        padded = F.pad(images, [pad, pad, pad, pad], mode='constant', value=pad_value)

    patches = F.unfold(padded, kernel_size=k)
    patches = patches.view(M, C_img * k * k, H_img, W_img)

    norms = (patches ** 2).sum(dim=1)

    centers = images

    return patches, norms, centers

```

```

@torch.no_grad()
def ls_vector_field(t, x, dataloader, k):
    b, c, h, w = x.shape
    at = (1 - t).to(x.device)
    bt = t.to(x.device)
    pad = k // 2

    x_padded = F.pad(x, [pad, pad, pad, pad], mode='circular')
    x_patches = F.unfold(x_padded, kernel_size=k).view(b, c * k * k, h, w)

    numerator = torch.zeros(b, c, h, w, device=x.device)
    denominator = torch.zeros(b, h, w, device=x.device)
    subtraction = None

    for images, labels in dataloader:
        images = images.to(x.device)
        M = images.shape[0]

        train_patches, train_norms, train_centers = extract_patches_grid(images, k, pad_mode="circular")
        dot = torch.einsum('bphw,mphw->bmhw', x_patches, train_patches)

        log_weights = (2 * at * dot - at**2 * train_norms[None]) / (2 * bt**2)

        values = x[:, None] - at * train_centers[None]

        numerator, denominator, subtraction = lse_accumulate(
            numerator, denominator, subtraction, log_weights, values
        )

    E_diff = numerator / denominator[:, None]
    vel = E_diff / bt - (x - E_diff) / at
    return vel

```

```

print("LS")

x_ls = flow_euler_solver(
    model=ls_vector_field,
    dataloader=dataloader,
    scales=SCALES,
    x_1=x_1
)

x_ls_img = denormalize(x_ls[0]).cpu().squeeze().clamp(0, 1)
plt.imshow(x_ls_img, cmap='gray')
plt.axis('off')
plt.title("LS")
plt.show()

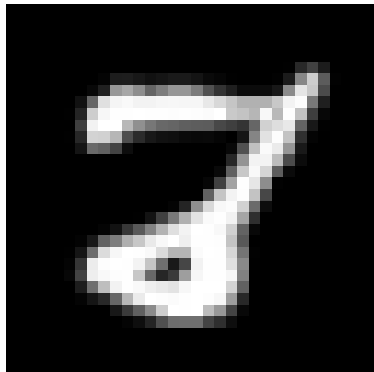
```

```

LS
100%|██████████| 19/19 [00:12<00:00, 1.53it/s]

```

LS



Note that for MNIST in particular, since digits are centered LS might work particularly well, although this isn't true in general.

ELS

The Equivariant Local Score (ELS) machine further recognizes the equivariant inductive bias from convolutions and attempts to model it with any other patch not necessarily centered at the same pixel location.

Since the ELS machine now compares the patch to every other in the training dataset independently of where it's centered, you might have to modify your previous implementation of the function responsible of extracting the patches.

```

@torch.no_grad()
def extract_patches(images, k):
    M, C_img, H_img, W_img = images.shape
    pad = k // 2
    padded = F.pad(images, [pad, pad, pad, pad], mode='circular')
    patches = F.unfold(padded, kernel_size=k)
    patches = patches.permute(0, 2, 1).reshape(M * H_img * W_img, C_img * k * k)
    norms = (patches ** 2).sum(dim=1)
    centers = images.permute(0, 2, 3, 1).reshape(M * H_img * W_img, C_img)
    return patches, norms, centers

```

```

@torch.no_grad()
def els_vector_field(t, x, dataloader, k):
    b, c, h, w = x.shape
    at = (1 - t).to(x.device)
    bt = t.to(x.device)
    pad = k // 2

```

```

x_padded = F.pad(x, [pad, pad, pad, pad], mode='circular')
x_patches = F.unfold(x_padded, kernel_size=k)
x_patches = x_patches.permute(0, 2, 1)

numerator = torch.zeros(b, c, h, w, device=x.device)
denominator = torch.zeros(b, h, w, device=x.device)
subtraction = None

for images, labels in dataloader:
    images = images.to(x.device)

    train_patches, train_norms, train_centers = extract_patches(images, k)
    P = train_patches.shape[0]

    dot = torch.matmul(x_patches, train_patches.T)

    log_weights = (2 * at * dot - at**2 * train_norms[None, None, :]) / (2 * bt**2)
    log_weights = log_weights.permute(0, 2, 1).view(b, P, h, w)

    values = x[:, None] - at * train_centers[None, :, :, None, None]

    numerator, denominator, subtraction = lse_accumulate(
        numerator, denominator, subtraction, log_weights, values
    )

E_diff = numerator / denominator[:, None]
vel = E_diff / bt - (x - E_diff) / at
return vel

```

```

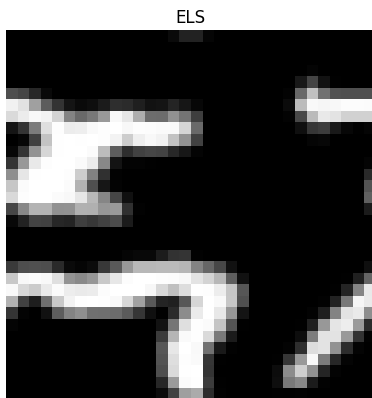
print("ELS")

x_els = flow_euler_solver(
    model=els_vector_field,
    dataloader=dataloader,
    scales=SCALES,
    x_1=x_1
)

x_els_img = denormalize(x_els[0]).cpu().squeeze().clamp(0, 1)
plt.imshow(x_els_img, cmap='gray')
plt.axis('off')
plt.title("ELS")
plt.show()

```

ELS
100%|██████████| 19/19 [00:18<00:00, 1.01it/s]



bbELS

The Boundary-Broken Equivariant Local Score (bbELS) machine accounts for the fact that patches near image borders contain location information. Border patches are compared only with training patches at similar relative positions to the boundary, while interior patches remain fully equivariant and are compared with all patches of the same shape.

To reflect this, patch extraction and matching near the boundaries must respect boundary structure. We enforce this by utilizing fixed padding instead of circular padding, so border patches are only matched to training patches with the same boundary overlap pattern.

You will now modify the patch extraction and vector field to implement the boundary-broken variant. The boundary constraint can be enforced or approximated in different ways, you are free to choose a reasonable implementation.

```

@torch.no_grad()
def extract_patches_padded(images, k, pad_value=0.0):
    M, C_img, H_img, W_img = images.shape
    pad = k // 2
    padded = F.pad(images, [pad, pad, pad, pad], mode='constant', value=pad_value)
    patches = F.unfold(padded, kernel_size=k)
    patches = patches.permute(0, 2, 1).reshape(M * H_img * W_img, C_img * k * k)
    norms = (patches ** 2).sum(dim=1)
    centers = images.permute(0, 2, 3, 1).reshape(M * H_img * W_img, C_img)
    return patches, norms, centers

```

```

@torch.no_grad()
def bbels_vector_field(t, x, dataloader, k):
    b, c, h, w = x.shape
    at = (1 - t).to(x.device)
    bt = t.to(x.device)
    pad = k // 2

    x_padded = F.pad(x, [pad, pad, pad, pad], mode='constant', value=0.0)

```

```

x_patches = F.unfold(x_padded, kernel_size=k)
x_patches = x_patches.permute(0, 2, 1)

ones = torch.ones(1, c, h, w, device=x.device)
ones_padded = F.pad(ones, [pad, pad, pad, pad], mode='constant', value=0.0)
mask_patches = F.unfold(ones_padded, kernel_size=k)
mask_patches = mask_patches.permute(0, 2, 1)

x_patches = x_patches * mask_patches

numerator = torch.zeros(b, c, h, w, device=x.device)
denominator = torch.zeros(b, h, w, device=x.device)
subtraction = None

for images, labels in dataloader:
    images = images.to(x.device)

    train_patches, train_norms, train_centers = extract_patches_padded(image)
    P = train_patches.shape[0]

    dot = torch.matmul(x_patches, train_patches.T)

    log_weights = (2 * at * dot - at**2 * train_norms[None, None, :]) / (2 *
    log_weights = log_weights.permute(0, 2, 1).view(b, P, h, w)

    values = x[:, None] - at * train_centers[None, :, :, None, None]

    numerator, denominator, subtraction = lse_accumulate(
        numerator, denominator, subtraction, log_weights, values
    )

E_diff = numerator / denominator[:, None]
vel = E_diff / bt - (x - E_diff) / at
return vel

```

```

print("bbELS")

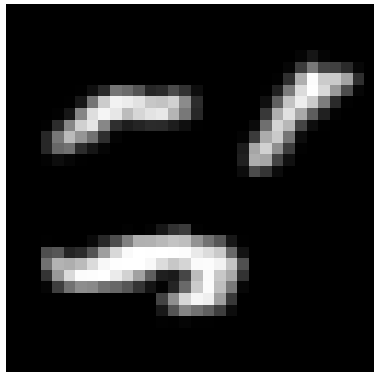
x_bbels = flow_euler_solver(
    model=bbels_vector_field,
    dataloader=dataloader,
    scales=SCALES,
    x_1=x_1
)

x_bbels_img = denormalize(x_bbels[0]).cpu().squeeze().clamp(0, 1)
plt.imshow(x_bbels_img, cmap='gray')
plt.axis('off')
plt.title("bbELS")
plt.show()

```

bbELS
100%|██████████| 19/19 [00:18<00:00, 1.02it/s]

bbELS



(Optional) Note that you could do a conditioned machine to mimic a conditioned UNet by filtering patches by label.

xD

Double-click (or enter) to edit